Metodologie di Programmazione (M-Z)

II semestre - a.a. 2023 – 2024

Parte 7 – Interfacce**

a cura di Mattia Samory*



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Letture

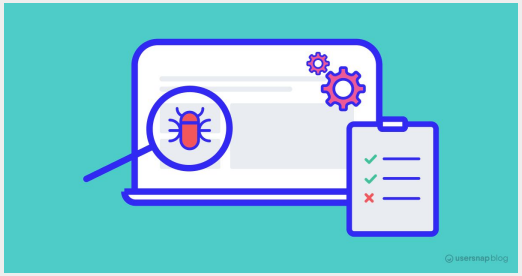
- Interfacce (Horstmann 10.1–10.2)

🔄 Recap

- testing

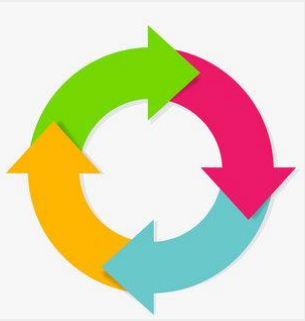
Test-Driven Development

proma i test, poi codice che passa i test



Iterative Development

compila spesso, esegui spesso, testa spesso

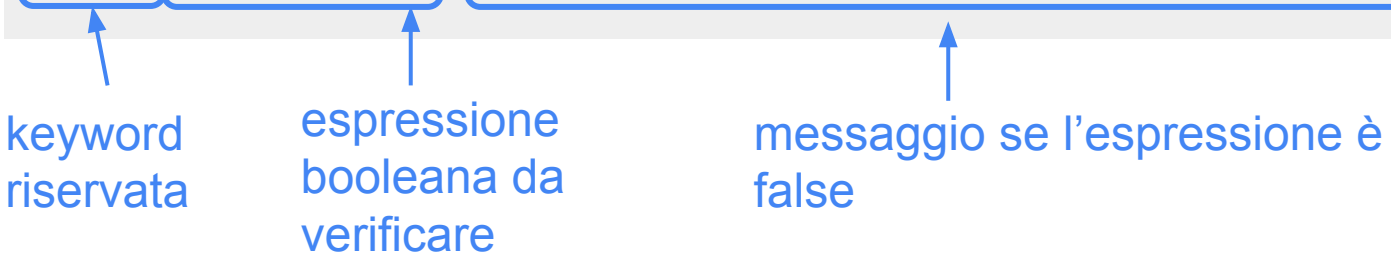


Assert verifica che una condizione viene valutata come vera ad un certo punto dell'esecuzione del programma.

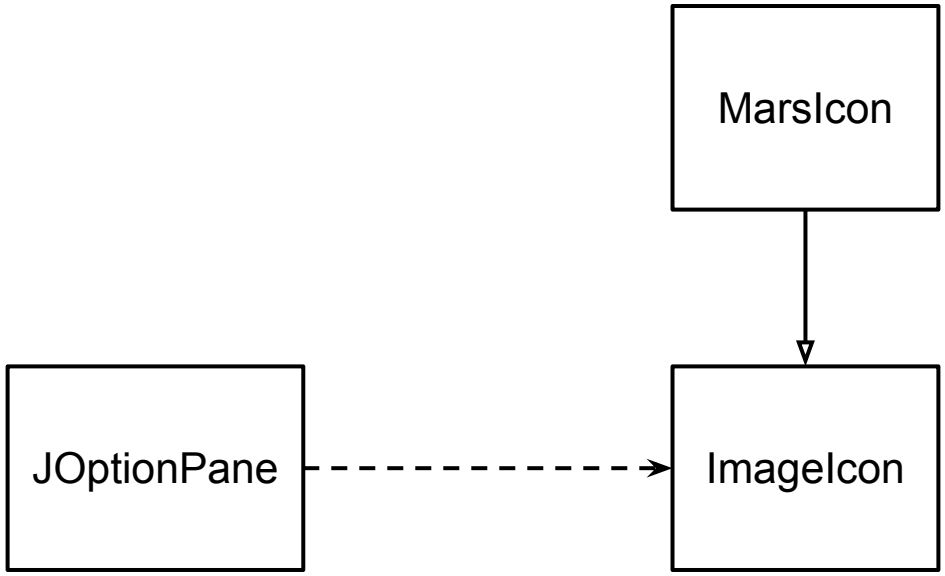
Esempio in Java: (va abilitato il controllo con lo switch ``java -ea IlProgramma'')

```
assert true == true : "Should never see this message…";

assert true == false : "Bad! True does not equal false.";
```

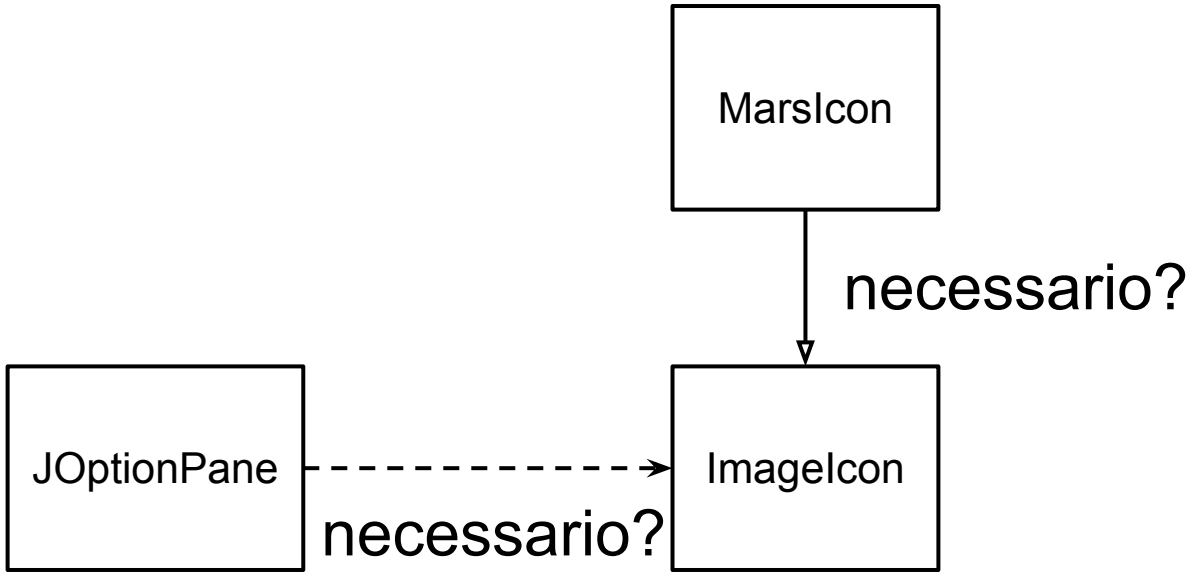


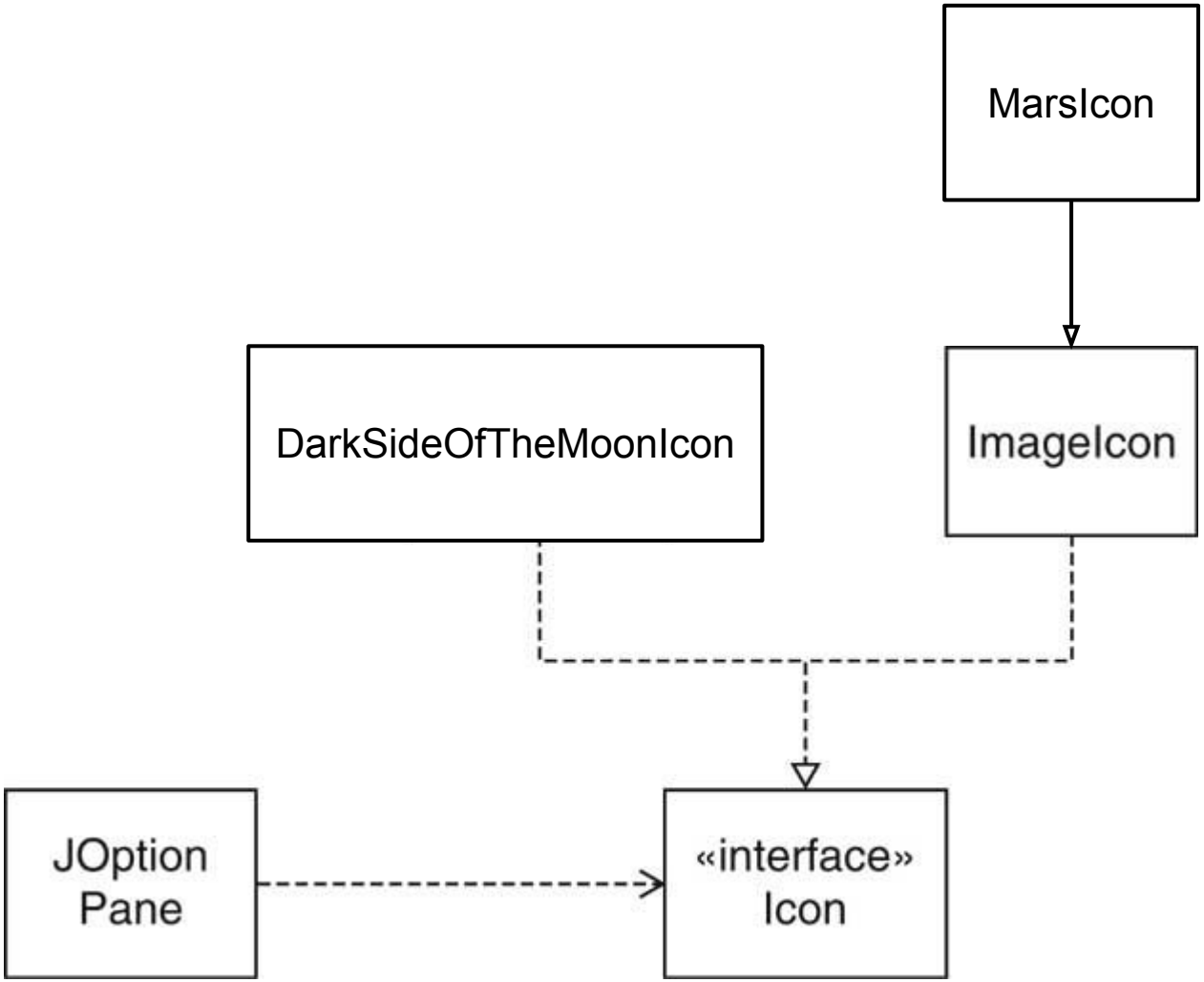
→ **eclipse**


MarsIcon
JOptionPane
ImageIcon

Benefici del polimorfismo

- Disaccoppiamento/Loose coupling
  - showMessageDialog non dipende da ImageIcon
  - non ha bisogno di sapere come processare immagini
- Estensibilità
  - L'utente può introdurre nuovi tipi di icona


MarsIcon
necessario?
JOptionPane
ImageIcon
necessario?


MarsIcon
DarkSideOfTheMoonIcon
ImageIcon
JOption
Pane
«interface»
Icon

🎯Obiettivi

- interfacce in java
- Iterable ed Iterator
- il problema💎

Le interfacce

http://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html

Le interfacce in Java

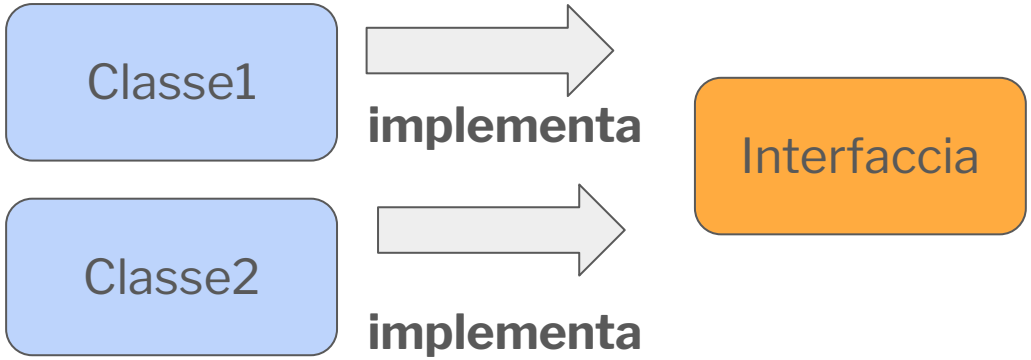
- Le interfacce sono uno strumento che Java mette a disposizione per consentire a più classi di fornire e implementare un insieme di metodi comuni
- Le interfacce definiscono e standardizzano l'interazione fra oggetti tramite un insieme limitato di operazioni





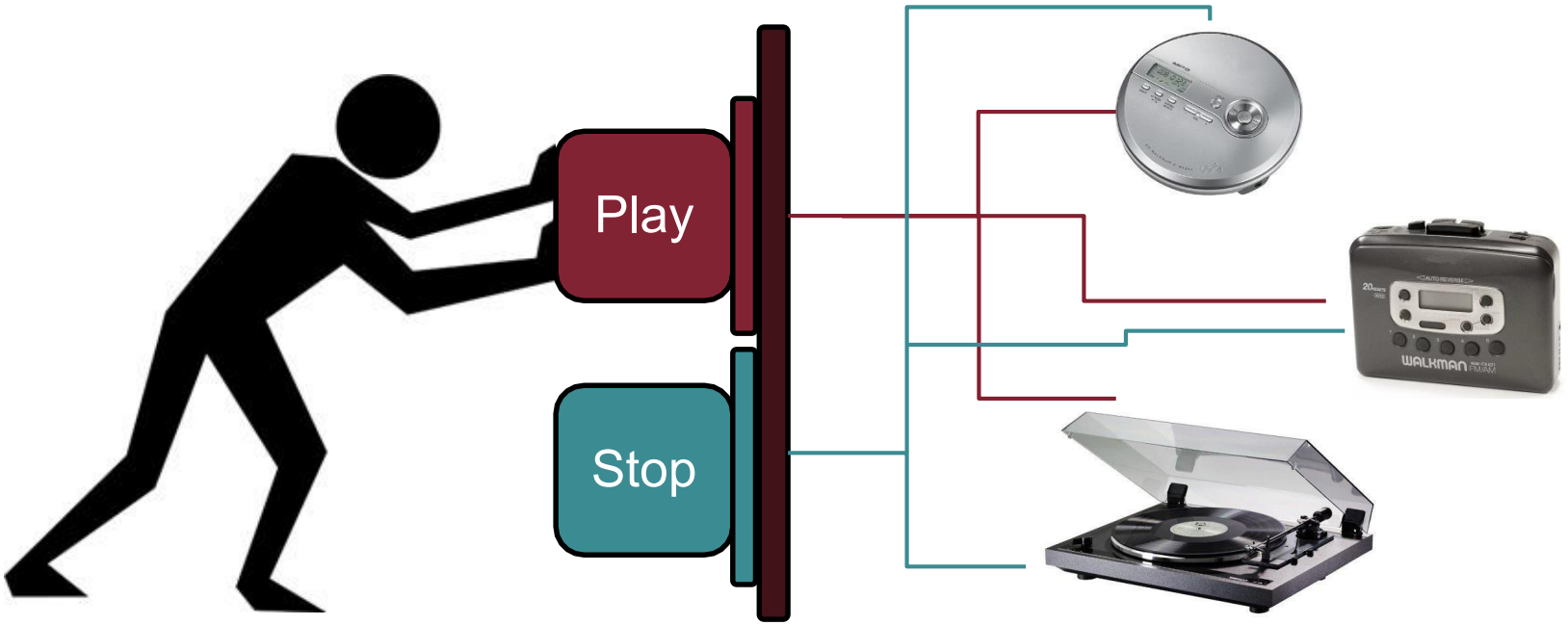


Oltre al sottoclasse, Java supporta un altro meccanismo di ereditarietà chiamato **sottotipizzazione**. La sottotipizzazione ci permette di specificare una relazione tra classi altrimenti non correlate, specificando un'**interfaccia**, **un insieme di metodi comuni** che ogni classe implementante deve contenere.



Le interfacce in Java

- Esse specificano soltanto il **comportamento** che un certo oggetto deve presentare all'esterno, cioè **cosa** quell'oggetto può fare
- L'**implementazione** di tali operazioni, cioè **come** queste vengono tradotte e realizzate, rimane invece non definito



Le interfacce in uno slogan

Le interfacce sono classi astratte al 100%*

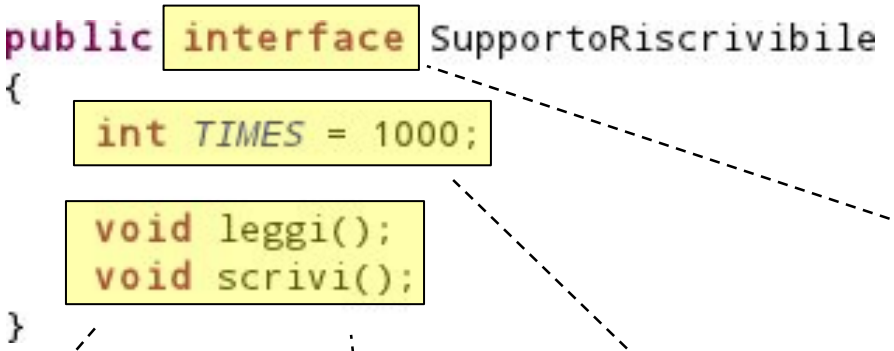


* se non definiamo metodi di default – da Java 8 in poi

Dichiarazione di un'interfaccia (1)

- Tutti i **metodi** dichiarati in un'interfaccia sono implicitamente **public abstract**
- Tutti i **campi** dichiarati in un'interfaccia sono implicitamente **public static final**

Dichiarazione di un'interfaccia (2)



Parola chiave interface: indica la definizione di una nuova interfaccia

Costante: implicitamente final, public e static

Dichiarazione di metodi: anche se non esplicitato, sono public e abstract

E' vietato definire qualunque implementazione

Implementare un'interfaccia (1)

```
public class Nastro implements SupportoRiscrivibile
{
    private Pellicola pellicola;

    @Override
    public void leggi()
    {
        attivaTestina();
        muoviTestina();
    }

    @Override
    public void scrivi()
    {
        attivaTestina();
        caricaTestina();
        muoviTestina();
        scaricaTestina();
    }

    public void attivaTestina() {}
    public void caricaTestina() {}
    public void scaricaTestina() {}
    public void muoviTestina() {}
```

- Per realizzare un'interfaccia è necessario che una classe la **implementi**, tramite la parola chiave **implements**
- Una classe che implementa una interfaccia decide di voler **esporre** pubblicamente all'esterno il comportamento descritto dall'interfaccia
- E' obbligatorio che ciascun metodo abbia esattamente la stessa intestazione che esso presenta nell'interfaccia



Implementare un'interfaccia (2)

```
public class MemoriaUsb implements SupportoRiscrivibile
{
    private CellaMemoria[] celle;

    @Override
    public void leggi()
    {
        // Leggi la cella corretta
    }

    @Override
    public void scrivi()
    {
        // Modifica la cella corretta
    }
}
```

Anche la classe MemoriaUsb implementa l'interfaccia SupportoRiscrivibile, definendo i propri metodi leggi() e scrivi()



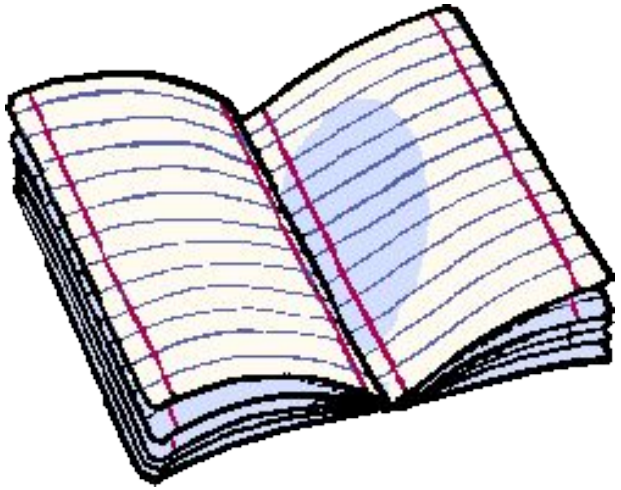
Implementare un'interfaccia (3)

```
public class Quaderno implements SupportoRiscrivibile
{
    private Foglio[] fogli;

    @Override
    public void leggi()
    {
        // Leggi la pagina corrente
    }

    @Override
    public void scrivi()
    {
        // Scrivi sulla pagina corrente
    }
}
```

Perfino un quaderno può essere visto come un supporto che è possibile leggere e scrivere!



Le interfacce permettono di modellare comportamenti comuni a classi che **non sono necessariamente** in relazione gerarchica (**is-a**, **è-un**)

Dichiarazione di un'interfaccia v2

- Un'interfaccia è una classe che può contenere soltanto
  - Costanti
  - Metodi astratti
  - Java 8: Implementazione di **default** di metodi e metodi **statici** (utilità)
  - Java 9: metodi **privati** tipicamente da invocare in metodi di default
- Tutti i **metodi** dichiarati in un'interfaccia sono implicitamente **public abstract**
- Tutti i **campi** dichiarati in un'interfaccia sono implicitamente **public static final**
- Tranne nel caso dei metodi di default o statici, non è possibile specificare alcun dettaglio implementativo
  - non vi è alcun corpo di metodo o variabile di istanza

Metodi di default e metodi statici (Java 8)

- E' possibile specificare delle implementazioni di default di metodi non statici
  - Mediante la parola chiave default
  - La motivazione principale per i metodi di default nelle interfacce è dovuta all'estensione di interfacce rilasciate in precedenza con nuovi metodi **senza rompere il contratto con il codice che utilizza le versioni precedenti**
- E implementazioni di metodi statici
  - Ricordate: i metodi statici **non godono del polimorfismo**
  - Sono metodi di utilità non associati alle singole istanze

Implementare un'interfaccia: il contratto (1)

- Implementare un'interfaccia equivale a **firmare un contratto** con il compilatore che stabilisce l'impegno ad implementare tutti i metodi specificati dall'interfaccia o a dichiarare la classe abstract



Implementare un'interfaccia: il contratto (2)

- 3 possibilità per una classe che implementa un'interfaccia:
  - fornire un'implementazione concreta di tutti i metodi, definendone il corpo
  - fornire l'implementazione concreta solo per un sottoinsieme proprio dei metodi dell'interfaccia
  - decidere di non fornire alcuna implementazione concreta
- Negli ultimi due casi, però, la classe va dichiarata abstract

Interfaccia vs API

- Un'interfaccia è un pezzo di codice.
  - È scritta in un file .java e deve essere compilata.
  - Definisce un "contratto" che le altre classi che implementano l'interfaccia devono seguire.
  - È un tipo di riferimento (proprio come una classe).
- Un'interfaccia di programmazione delle applicazioni (API) comprende l'intero insieme di funzionalità accessibili fornite da una libreria, un framework o un servizio.
  - È pensata per essere leggibile da un utente umano.
  - Può includere più interfacce, classi e/o metodi

Metodi privati (Java 9)

- Per facilitare il riuso del codice, da Java 9 è possibile definire metodi privati all'interno di un'interfaccia

```
public interface Loggabile
{
    private void log(String msg, LogLevel level)
    {
        // Codice per il logging
    }

    default void logInfo(String msg) { log(msg, LogLevel.INFO); }
    default void logWarn(String msg) { log(msg, LogLevel.WARNING); }
    default void logError(String msg) { log(msg, LogLevel.ERROR); }
    default void logFatal(String msg) { log(msg, LogLevel.FATAL); }
}
```

🎯Obiettivi

- ✅ interfacce in java
- ❑ Iterable ed Iterator
- ❑ il problema💎

👐TPS

- Pensate a tre classi che realizzano diversi tipi di successioni
  - La successione $\{ i^2 \}$ (es. 0, 1, 4, 9, 16, ecc.)
  - La successione casuale (es. -42, 2, 5, 18, 154, ecc.)
  - La successione di Fibonacci (es. 1, 1, 2, 3, 5, 8, 13, ecc.)
  - La successione infinita 4 8 15 16 23 42 4 8 15 16 23 42 ecc.
- Elaborare un meccanismo generale per la generazione della successione indipendentemente dall'implementazione specifica

Un esempio: l'interfaccia Iterabile (1)

- Ci sono molte classi di **natura diversa** che rappresentano sequenze di elementi
  - Es. array, liste, stringhe, ecc.
- Tuttavia, le sequenze hanno qualcosa in comune: è possibile iterare sui loro elementi:

```
public interface Iterabile
{
    boolean hasNext();
    Object next();
    void reset();
}
```

Un esempio: l'interfaccia Iterabile (2)

- Ciascuna classe implementerà i metodi **a suo modo**:

```
public class MyIntegerArray implements Iterabile
{
    private Integer[] array;
    private int k = 0;

    public MyIntegerArray(Integer[] array)
    {
        this.array = array;
    }

    @Override
    public boolean hasNext()
    {
        return k < array.length;
    }

    @Override
    public Object next()
    {
        return array[k++];
    }

    @Override
    public void reset() { k = 0; }
}
```

```
public class MyString implements Iterabile
{
    private String s;
    private int k = 0;

    public MyString(String s)
    {
        this.s = s;
    }

    @Override
    public boolean hasNext()
    {
        return k < s.length();
    }

    @Override
    public Object next()
    {
        return s.charAt(k++);
    }

    @Override
    public void reset() { k = 0; }
}
```

Possono essere specializzati a **Integer** e **Character**

QUESTA SOLUZIONE NON VA BENE!!!!!!

Un esempio: l'interfaccia Iterabile (2)

- Infine, testiamo le due classi:

```
public class InterfacceChePassione
{
    static public void main(String[] args)
    {
        Iterabile i1 = new MyIntegerArray(new Integer[] {10, 20, 30, 40});
        Iterabile i2 = new MyString("abcdefghi");

        for (Iterabile i : new Iterabile[] { i1, i2 })
        {
            while(i.hasNext())
                System.out.println(i.next());
        }
    }
}
```

Assegniamo un oggetto di una classe che implementa l'interfaccia a una variabile-riferimento a interfaccia

Possiamo creare un array di riferimenti a interfaccia!

Abbiamo ora un meccanismo generale per iterare su **sequenze** che implementano l'interfaccia

L'esempio "Iterabile" non è la soluzione ideale per iterare su una collezione

- Perché?
- **Non permette** di mantenere contatori multipli sullo stesso oggetto
- Il seguente esempio **NON FUNZIONA**!

```
MyString s = new MyString("ciao");
while(s.hasNext())
{
        char c1 = s.next();
        while(s.hasNext())
        {
                char c2 = s.next();
                System.out.println(c1+"-"+c2);
        }
}
```

La soluzione: usare le interfacce Iterable e Iterator

- Interfacce standard di Java che disaccoppiano l'oggetto su cui iterare dall'oggetto che tiene la posizione d'iterazione
- java.lang.Iterable:

Method Summary

Methods

Modifier and Type	Method and Description
`Iterator<T>`	**`iterator()`** Returns an iterator over a set of elements of type T.

- java.util.Iterator

Method Summary

Methods

Modifier and Type	Method and Description
`boolean`	**`hasNext()`** Returns `true` if the iteration has more elements.
`E`	**`next()`** Returns the next element in the iteration.
`void`	**`remove()`** …es from the underlying collection the last element returned by this iterator (optional operation).



L'interfaccia java.util.Iterator<E>

- E' un'interfaccia fondamentale che permette di **iterare** su collezioni
- Espone tre metodi:

Metodo	Descrizione
boolean **hasNext**()	Restituisce true se esiste ancora un successivo elemento nella collezione
E **next**()	Restituisce l'elemento successivo
void **remove**()	Rimuove l'elemento corrente (operazione **opzionale**)

- E' in relazione con l'interfaccia Iterable nel senso che chi implementa Iterable restituisce un Iterator sull'oggetto-collezione



Esercizio: il mio array iterabile

- Scrivere una classe che implementa un array di interi iterabile
- Utilizzare l'interfaccia Iterable<Integer>

Esercizio: la mia stringa iterabile

- Scrivere una classe che implementa una stringa iterabile utilizzando Iterable<Character>

Esercizio: successioni con iterazione standard

- Implementare l'esercizio sulle successioni utilizzando il meccanismo standard di Java per l'iterazione



Esercizio: Lista linkata di interi

- Implementare una lista linkata di interi in modo che implementi l'interfaccia java.util.List e rendendo la lista iterabile con il costrutto for each di Java
  - Se si sono viste le classi interne, implementare l'iterazione mediante classi interne

Esempio: Il Jukebox






```java
import java.util.ArrayList;
import java.util.Iterator;

/**
 * Rappresenta un JukeBox di canzoni su cui si può iterare
 *
 * @author navigli
 */
public class Jukebox implements Iterable<Canzone>
{
    /**
     * Elenco delle canzoni
     */
    private ArrayList<Canzone> canzoni = new ArrayList<Canzone>();

    /**
     * Permette di aggiungere una canzone
     * @param c la canzone da aggiungere
     */
    public void addCanzone(Canzone c)
    {
        canzoni.add(c);
    }

    @Override
    public Iterator<Canzone> iterator()
    {
        return canzoni.iterator();
    }
}
```

🎯Obiettivi

- ✅ interfacce in java
- ✅ Iterable ed Iterator
- ❑ il problema💎

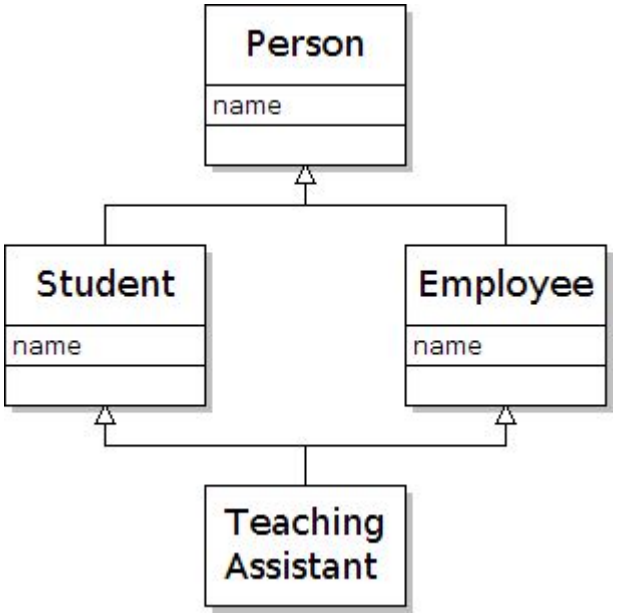
Interfacce vs. classi astratte

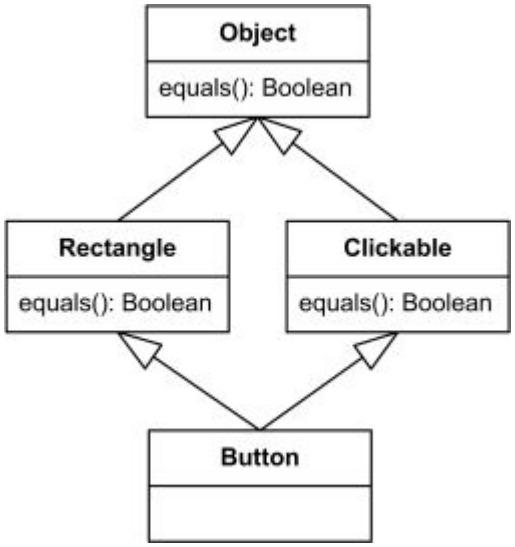
- **Una domanda nasce spontanea:** Se implementando un'interfaccia **devo** dichiarare tutti i metodi in essa definiti, perché non ricorrere ad una classe astratta?



Il problema del diamante

- Con l'ereditarietà multipla si possono creare situazioni **poco chiare** di duplicazione di metodi e campi





Un altro esempio di problema del diamante





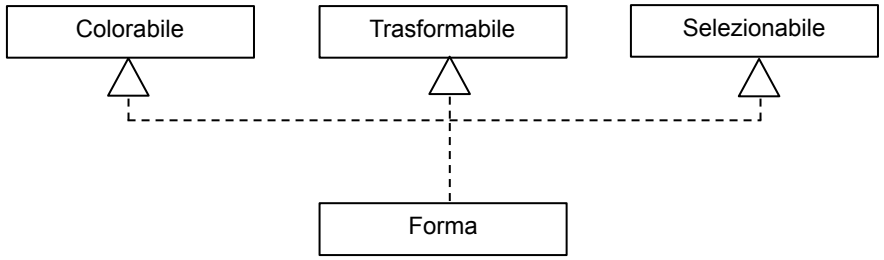






Ereditarietà multipla (1)

- In Java non è consentito estendere più di una classe alla volta: ovvero extends può essere seguito **solo da un unico nome di classe**
- Al contrario, una classe può implementare tutte le interfacce desiderate



- E' inoltre possibile estendere **1** classe e contemporaneamente implementare **n** interfacce diverse

Ereditarietà multipla (2)

```
public class Forma implements Colorabile, Trasformabile, Selezionabile
{

    @Override
    public void seleziona()
    {
    }

    @Override
    public void deseleziona()
    {
    }

    @Override
    public void sposta(Point point)
    {
    }

    @Override
    public void ruota(Angle angle)
    {
    }

    @Override
    public void ritaglia(Point upperLeft, Point bottoRight)
    {
    }

    @Override
    public void scala(double factor)
    {
    }

    @Override
    public void colora(Color c)
    {
    }

    @Override
    public void rimuoviColore()
    {
    }
}
```

La classe Forma implementa **3 interfacce** e definisce tutti i metodi da esse dichiarati

```
public interface Selezionabile
{
    void seleziona();
    void deseleziona();
}
```

```
public interface Colorabile
{
    void colora(Color c);
    void rimuoviColore();
}
```

```
public interface Trasformabile
{
    void sposta(Point point);
    void ruota(Angle angle);
    void ritaglia(Point upperLeft, Point bottoRight);
    void scala(double factor);
}
```

Relazione tra interfacce e classi che le implementano

- Nel momento in cui una classe C decide di implementare un'interfaccia I, tra queste due classi si instaura una relazione di tipo **is-a**, ovvero C è di tipo I
- Comportamento simile a quello dell'ereditarietà
- Quindi anche per le interfacce valgono le regole del polimorfismo
- Ad esempio, la seguente dichiarazione è lecita:

```
SupportoRiscrivibile supporto = new Nastro();
supporto.leggi();
```

- E ci consente di usare l'oggetto della classe Nastro **come fosse di tipo SupportoRiscrivibile**
  - Con conseguente **restringimento della visibilità** ai soli metodi dell'interfaccia SupportoRiscrivibile

Cosa succede se due metodi di default vengono ereditati da due interfacce implementate?

- E' necessario implementare quel metodo nella classe che implementa le due interfacce e "disambiguare" il metodo chiamando quello/i appropriato/i con la sintassi

  Interfaccia.super.metodo(…);

Interfacce notevoli

Interfaccia	Descrizione
Comparable	Impone un ordinamento naturale degli oggetti tramite il metodo: int compareTo(T b), che restituisce un valore >, = o < 0 se l'oggetto è rispettivamente maggiore, uguale o minore di b (T è un tipo generico; può essere qualsiasi tipo)
Cloneable	Le classi che implementano quest'interfaccia dichiarano al metodo clone() di Object che è legale effettuare una copia campo-a-campo delle istanze della classe Il metodo clone() invocato su oggetti di classi che non implementano Cloneable solleva una CloneNotSupportedException
Serializable	Quest'interfaccia non possiede metodi o campi e serve soltanto ad identificare il fatto che l'oggetto è serializzabile, cioè memorizzabile su un certo supporto



Esercizio: Animali

- Progettare una gerarchia di classi dei seguenti animali, ciascuno con determinate caratteristiche:
  - Uccello (vola, becca)
    - Pinguino (becca, nuota)
    - Aquila (vola, becca)
  - Pesce (nuota)
    - Pesce volante (nuota, vola)
  - Cane (salta, corre, fedele a, domestico)
  - Felino (salta, corre, fa le fusa)
    - Gatto (salta, corre, fa le fusa, domestico)
  - Uomo (salta, corre, pensa, nuota, vola?)

Passare funzioni in input mediante le interfacce

- Le interfacce permettono il passaggio in input di funzioni con una determinata intestazione
- Ad esempio:

```
public interface Runnable
{
    void run();
}
```

Method Summary

All Methods | **Instance Methods** | **Abstract Methods**

Modifier and Type	Method and Description
`void`	**`run()`** When an object implementing interface `Runnable` is used to create a thread, starting the thread causes the object's `run` method to be called in that separately executing thread.

Passare funzioni in input mediante le interfacce

- Posso quindi implementare Runnable in quante classi voglio, **definendo diversi comportamenti** per il metodo void run()

```
public class Parlare implements Runnable
{
   public void run() { System.out.println("bla bla bla..."); }
}

public class Dormire implements Runnable
{
   public void run() { System.out.println("zzzzz....."); }
}
```

Le enumerazioni possono estendere interfacce

- A cosa serve? Posso rendere le **enumerazioni estensibili quanto voglio**!
- Nell'esercizio sulle espressioni matematiche, posso utilizzare costanti enumerative come istanziazioni dell'interfaccia OperatoreBinario

```
public interface OperatoreBinario
{
    double applica(double a, double b);
}

public enum OperatoriDiBase implements OperatoreBinario
{
    SOMMA { public double applica(double a, double b) { return a+b; } },
    SOTTRAZIONE { public double applica(double a, double b) { return a-b; } },
    PRODOTTO { public double applica(double a, double b) { return a*b; } },
    DIVISIONE { public double applica(double a, double b) { return a/b; } }
}
```

Sovrascrivere il metodo clone (1)

- L'operatore di assegnazione = **non** effettua una copia dell'oggetto, ma solo del riferimento all'oggetto
- Per creare una copia di un oggetto è necessario richiamare clone()
- x.clone() != x **sarà sempre vero**
- clone **non richiama il costruttore della classe**
- Tuttavia l'implementazione nativa di default di Object.clone copia l'oggetto **campo per campo** (shallow copy)
  - **Ottimo** se i campi sono tutti **primitivi**
  - **Problematico** se i campi sono **riferimenti**

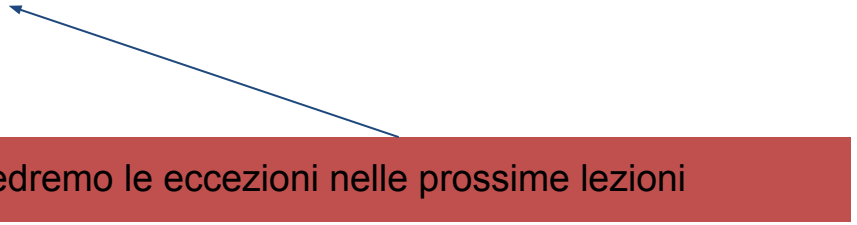
Sovrascrivere il metodo clone (2)

- Per implementare la copia in una propria classe è necessario sovrascrivere clone() che è

  PROTETTA (quindi visibile solo in gerarchia e nel package)

  – Se il nostro oggetto contiene riferimenti e vogliamo evitare che la copia contenga un riferimento allo stesso oggetto membro, non possiamo chiamare semplicemente (o non chiamiamo proprio) super.clone()

- E' necessario implementare l'interfaccia "segnaposto" Cloneable altrimenti Object.clone emetterà semplicemente l'eccezione CloneNotSupportedException



Clone: attenzione alla "shallow copy"!

```
public class IntVector implements Cloneable
{
    ArrayList<Integer> list = new ArrayList<>();

    public IntVector(int... values)
    {
        for (int v : values) list.add(v);
    }

    public void add(int v) { list.add(v); }

    public IntVector getCopy()
    {
        try
        {
            return (IntVector)clone();
        }
        catch(CloneNotSupportedException e) { return null; }
    }
}
```

Clone: attenzione alla "shallow copy"!

- Testiamo la classe con Junit:

```
import static org.junit.Assert.*;

public class IntVectorTest
{
    @Test
    public void testGetCopy1()
    {
        IntVector v = new IntVector(4, 8, 15, 16, 23, 42);
        IntVector v2 = v.getCopy();

        assertNotSame(v.list, v2.list);
    }

    @Test
    public void testGetCopy2()
    {
        IntVector v = new IntVector(4, 8, 15, 16, 23);
        IntVector v2 = v.getCopy();
        v.add(42);

        assertNotEquals(v.list, v2.list);
    }
}
```

I due riferimenti puntano alla stessa identica lista!

Sovrascrivere il metodo clone (3)

- Per evitare la copia dei riferimenti, è necessaria la clonazione "profonda" (deep cloning)
  - Si può usare Object.clone per la clonazione dei tipi primitivi
  - E richiamare .clone() su tutti i campi che sono riferimenti ad altri oggetti, impostando i nuovi riferimenti nell'oggetto clonato
- Con il **deep cloning**, i test Junit hanno successo!

```
public IntVector getCopy()
{
    try
    {
        IntVector v = (IntVector)clone();
        v.list = (ArrayList<Integer>)list.clone();
        return v;
    }
    catch(CloneNotSupportedException e) { return null; }
}
```

🎯Obiettivi

- ✅ interfacce in java
- ✅ Iterable ed Iterator
- ✅ il problema💎

Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.